

Neural Networks by Hand

Learning goals

After this exercise, you should be able to

- evaluate a simple neural network by hand,
- explain the role of weights, biases, and activation functions,
- perform a complete forward pass through a small neural network,
- calculate the mean squared error for a small training dataset,
- use backpropagation to compute gradients of the loss with respect to all trainable parameters,
- explain how the ReLU activation affects gradient propagation,
- perform one gradient-descent update,
- verify that a gradient step can reduce the training loss,
- distinguish between fitting the training points and learning the underlying function,
- explain why a ReLU network produces a piecewise-linear function.

The calculations in this sheet are designed to be done by hand.

Part I - A small neural network

We want to approximate the function

$$f(x) = -x^2 + 4x$$

We use only the four training inputs

$$x_i = 0, 1, 2, 3.$$

Exercise 1.1 - Generate the training data

Evaluate the function at the four training inputs and complete the table.

x_i	$y_i = f(x_i)$
0	
1	
2	
3	

Plot the four training points and sketch the true function $f(x) = -x^2 + 4x$ in the range $-1 \leq x \leq 5$.

Exercise 1.2 - Our neural network architecture

We use the architecture

$$1 \rightarrow 2 \rightarrow 1.$$

There is

- one input
- one hidden layer with two neurons
- one output neuron

The hidden layer uses the *ReLU* activation

$$\text{ReLU}(z) = \max(0, z)$$

The two hidden neurons are

$$z_{1,i} = w_1 x_i + b_1$$

$$h_{1,i} = \text{ReLU}(z_{1,i})$$

and

$$z_{2,i} = w_2 x_i + b_2$$

$$h_2 = \text{ReLU}(z_{2,i})$$

The output layer is linear:

$$\hat{y}_i = v_1 h_{1,i} + v_2 h_{2,i} + c$$

Sketch the following

- the *ReLU* function for input values $z \in [-5, 5]$
- the neural network

for this, count the trainable parameters first. Your drawing should contain

- the input x_i
- the two hidden neurons
- the output $\hat{y}_i$
- all weights
- all biases
- the ReLU activation in the hidden layer

Exercise 1.3 - A complete forward pass

To build the neural network, the parameters need to be fit. We start with a list of initial guesses for which the output of the neural network is calculated. This will be compared to the training data using the mean squared error. A real neural network training proceeds until the MSE is sufficiently small. In this exercise we will however only produce a single calculation.

For the first training step, use the following initial parameters:

$$w_1 = 1 \quad b_1 = -0.5$$

$$w_2 = -1 \quad b_2 = 2.5$$

$$v_1 = 1 \quad v_2 = 1 \quad c = 0$$

This exercise is divided into multiple steps:

Step 1 - Hidden-layer activations

For each of the four training inputs, calculate

$$z_{1,i} = w_1 x_i + b_1$$

$$h_{1,i} = \text{ReLU}(z_{1,i})$$

and analogous for $z_{2,i}$ and $h_{2,i}$. Complete the table below:

x_i	y_i	$z_{1,i}$	$h_{1,i}$	$z_{2,i}$	$h_{2,i}$
0	0				
1	3				
2	4				
3	3				

For each hidden neuron, identify for which input values the ReLU is active.

Step 2 - Output predictions

Using

$$\hat{y} = v_1 h_1 + v_2 h_2 + c$$

calculate the network prediction for every training point:

x	y	$\hat{y}$	residual $\hat{y} - y$
0	0	0	0
1	3	3	0
2	4	4	0
3	3	3	0

Plot the four predictions together with the four training points.

Step 3 - Calculate the loss

We use the mean squared error

$$L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

with

$$N = 4$$

Calculate the MSE for the current network. This is the loss before training.

Exercise 1.4 - Backpropagation by hand

We now calculate how the loss changes with respect to every trainable parameter.

The important idea is:

”Start at the loss and repeatedly apply the chain rule while moving backwards through the network”

We perform this calculation only once by hand. A neural-network library repeats exactly the same type of calculation automatically.

Step 1 - Start at the loss

For one training sample the loss is

$$L_i = (\hat{y}_i - y_i)^2$$

and so the mean loss is given by a sum

$$L = \frac{1}{4} \sum_i L_i$$

The gradient is given by

$$\frac{\partial L}{\partial \hat{y}_i} = \frac{2}{4}(\hat{y}_i - y_i)$$

One defines

$$\delta_i = \frac{\partial L}{\partial \hat{y}_i}$$

Calculate δ_i for every training point.

x_i	y_i	$\hat{y}_i$	$\hat{y}_i - y_i$	$\delta = \partial L / \partial \hat{y}$
0	0			
1	3			
2	4			
3	3			

Step 2 - Gradients of the output layer

Gradient with respect to v_1

The output is

$$\hat{y}_i = v_1 h_{1,i} + v_2 h_{2,i} + c$$

thus

$$\frac{\partial \hat{y}_i}{\partial v_1} = h_{1,i}$$

Use the chain rule to obtain

$$\frac{\partial L}{\partial v_1} = \sum_i \frac{\partial L}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial v_1}$$

Therefore,

$$\frac{\partial L}{\partial v_1} = \sum_i \delta_i h_{1,i}$$

Calculate this gradient.

Gradient with respect to v_2

Analogously, calculate

$$\frac{\partial L}{\partial v_2} = \sum_i \delta_i h_{2,i}$$

Gradient with respect to c

Since

$$\frac{\partial \hat{y}_i}{\partial c} = 1$$

calculate

$$\frac{\partial L}{\partial c} = \sum_i \delta_i$$

Record your results:

parameter	gradient
v_1	
v_2	
c	

Step 3 - Backpropagate through ReLU

The derivative of ReLU is

$$\text{ReLU}'(z) = \begin{cases} 0, & z < 0, \\ 1, & z > 0 \end{cases}$$

For the values encountered in this exercise, none of the z values are exactly zero.

Using your values from the forward pass, complete the table:

x_i	$z_{1,i}$	$\text{ReLU}'(z_{1,i})$	$z_{2,i}$	$\text{ReLU}'(z_{2,i})$
0				
1				

x_i	$z_{1,i}$	$\text{ReLU}'(z_{1,i})$	$z_{2,i}$	$\text{ReLU}'(z_{2,i})$
2				
3				

For the first hidden neuron,

$$\frac{\partial L}{\partial z_{1,i}} = \frac{\partial L}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial h_{1,i}} \frac{\partial h_{1,i}}{\partial z_{1,i}}$$

Because

$$\frac{\partial \hat{y}_i}{\partial h_{1,i}} = v_1$$

we obtain

$$\frac{\partial L}{\partial z_{1,i}} = \delta_i v_1 \text{ReLU}'(z_{1,i})$$

Calculate these four values.

Do the same for the second hidden neuron:

$$\frac{\partial L}{\partial z_{2,i}} = \delta_i v_2 \text{ReLU}'(z_{2,i})$$

Complete:

x_i	$\partial L / \partial z_{1,i}$	$\partial L / \partial z_{2,i}$
0		
1		
2		
3		

What happens to the gradient when a ReLU neuron is inactive?

Step 4 - Gradients of the first layer

For the first neuron,

$$z_{1,i} = w_1 x_i + b_1.$$

Therefore,

$$\frac{\partial z_{1,i}}{\partial w_1} = x_i$$

and

$$\frac{\partial z_{1,i}}{\partial b_1} = 1$$

Calculate the gradients

$$\frac{\partial L}{\partial w_1}$$

and

$$\frac{\partial L}{\partial b_1}$$

Repeat the calculation for the second hidden neuron and collect all seven gradients:

parameter	gradient
w_1	
b_1	
w_2	
b_2	
v_1	
v_2	
c	

Exercise 1.5 - One gradient-descent step

We now update every parameter using gradient descent. For a parameter θ

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \frac{\partial L}{\partial \theta}$$

Use the learning rate

$$\eta = 0.1$$

Step 1 - pdate the parameters

Complete the table.

parameter	old value	gradient	new value
w_1	1.0		
b_1	-0.5		
w_2	-1.0		
b_2	2.5		
v_1	1.0		
v_2	1.0		
c	0.0		

Step 2 - Did training improve the model?

Using the updated parameters, perform one more forward pass (see Exercise 1.3). (You do not need to perform another backward pass)

Calculate the new predictions and the new MSE.

x	y	new prediction $\hat{y}_{\text{new}}$
0	0	
1	3	
2	4	
3	3	

Compare

$$L_{\text{old}}$$

and

$$L_{\text{new}}.$$

Did this gradient-descent step reduce the loss? Does every single training point necessarily improve after one gradient step?

Explain why the optimizer can still reduce the total loss even if one individual prediction becomes worse.

Solutions - A small neural network

Solution 1.1

The training data are

x_i	y_i
0	0
1	3
2	4
3	3

The function is nonlinear, so a single affine model $\hat{y}_i = wx_i + b$ cannot reproduce all four points exactly.

Even if a model reproduces all four training points, this does not prove that it has learned the true function between or outside those points.

Solution 1.2

The trainable parameters are

$$w_1, b_1, w_2, b_2, v_1, v_2, c.$$

There are therefore 7 trainable parameters.

Solution 1.3

Using

$$w_1 = 1, \quad b_1 = -0.5$$

and

$$w_2 = -1, \quad b_2 = 2.5$$

we obtain

x	y	z_1	h_1	z_2	h_2
0	0	-0.5	0	2.5	2.5
1	3	0.5	0.5	1.5	1.5
2	4	1.5	1.5	0.5	0.5
3	3	2.5	2.5	-0.5	0

Because

$$v_1 = v_2 = 1, \quad c = 0$$

the predictions are

x	y	h_1	h_2	$\hat{y}$	$\hat{y} - y$
0	0	0	2.5	2.5	2.5
1	3	0.5	1.5	2.0	-1.0
2	4	1.5	0.5	2.0	-2.0
3	3	2.5	0	2.5	-0.5

Thus,

$$L = \frac{1}{4} (2.5^2 + (-1)^2 + (-2)^2 + (-0.5)^2)$$

so $L = 2.875$

Solution 1.4

For every sample,

$$\delta_i = \frac{\partial L}{\partial \hat{y}_i} = \frac{1}{2}(\hat{y}_i - y_i)$$

Therefore,

$$\delta = (1.25, -0.5, -1, -0.25)$$

For v_1 ,

$$\frac{\partial L}{\partial v_1} = \sum_i \delta_i h_{1,i} = 1.25(0) - 0.5(0.5) - 1(1.5) - 0.25(2.5)$$

and therefore

$$\frac{\partial L}{\partial v_1} = -2.375$$

For v_2 ,

$$\frac{\partial L}{\partial v_2} = 1.25(2.5) - 0.5(1.5) - 1(0.5) - 0.25(0)$$

so

$$\frac{\partial L}{\partial v_2} = 1.875$$

For the output bias,

$$\frac{\partial L}{\partial c} = 1.25 - 0.5 - 1 - 0.25 = -0.5$$

For the first hidden neuron,

$$z_1 = (-0.5, 0.5, 1.5, 2.5),$$

so

$$\text{ReLU}'(z_1) = (0, 1, 1, 1)$$

For the second hidden neuron,

$$z_2 = (2.5, 1.5, 0.5, -0.5)$$

so

$$\text{ReLU}'(z_2) = (1, 1, 1, 0)$$

Because initially

$$v_1 = v_2 = 1$$

we obtain

$$\frac{\partial L}{\partial z_1} = (0, -0.5, -1, -0.25)$$

and

$$\frac{\partial L}{\partial z_2} = (1.25, -0.5, -1, 0)$$

An inactive ReLU has derivative zero, so no gradient is propagated through that neuron for that sample.

For the first hidden neuron,

$$\frac{\partial L}{\partial w_1} = 0(0) + (-0.5)(1) + (-1)(2) + (-0.25)(3) = -3.25$$

Furthermore,

$$\frac{\partial L}{\partial b_1} = 0 - 0.5 - 1 - 0.25 = -1.75$$

For the second neuron,

$$\frac{\partial L}{\partial w_2} = 1.25(0) + (-0.5)(1) + (-1)(2) + 0(3) = -2.5$$

Finally,

$$\frac{\partial L}{\partial b_2} = 1.25 - 0.5 - 1 = -0.25$$

All gradients are therefore

parameter	gradient
w_1	-3.25
b_1	-1.75
w_2	-2.50
b_2	-0.25
v_1	-2.375
v_2	1.875
c	-0.50

Solution 1.5

With

$$\eta = 0.1$$

the updated parameters are

parameter	old value	gradient	new value
w_1	1.0	-3.25	1.325
b_1	-0.5	-1.75	-0.325
w_2	-1.0	-2.50	-0.750
b_2	2.5	-0.25	2.525
v_1	1.0	-2.375	1.2375
v_2	1.0	1.875	0.8125
c	0.0	-0.50	0.050

The new predictions are approximately

x	y	$\hat{y}_{\text{new}}$
0	0	2.102
1	3	2.730
2	4	3.760
3	3	4.790

The new loss is approximately

$$L_{\text{new}} \approx 1.94$$

Since

$$1.94 < 2.875$$

the gradient step has reduced the total loss.

The prediction for every individual point does not need to improve. Gradient descent follows the gradient of the combined loss over the complete dataset.

Part II - Investigating a fitted neural network

Repeating forward pass, backpropagation, and parameter updates many times is exactly what neural-network training does.

Instead of repeating these calculations by hand, we now inspect a network whose parameters are already chosen.

Consider

$$h_1(x) = \text{ReLU}(x),$$

$$h_2(x) = \text{ReLU}(x - 1.5),$$

and

$$\hat{f}(x) = 3h_1(x) - 4h_2(x).$$

This is again a network with architecture

$$1 \rightarrow 2 \rightarrow 1.$$

Exercise 2.1 - Identify the network parameters

Write the network in the standard form

$$z_1 = w_1x + b_1, \quad h_1 = \text{ReLU}(z_1),$$

$$z_2 = w_2x + b_2, \quad h_2 = \text{ReLU}(z_2),$$

$$\hat{y} = v_1h_1 + v_2h_2 + c.$$

Determine the parameters

$$w_1, b_1, w_2, b_2, v_1, v_2, c.$$

Exercise 2.2 - Check the training points

Evaluate the fitted network at

$$x = 0, 1, 2, 3.$$

Complete the table.

x	true $f(x)$	network $\hat{f}(x)$
0	0	
1	3	
2	4	
3	3	

Calculate the training MSE.

What is special about this result?

Exercise 2.3 - Has the network learned the true function?

Now evaluate both the true function and the neural network at new input values:

$$x = 0.5, 1.5, 2.5, 4.$$

Complete the table.

x	true $f(x)$	network $\hat{f}(x)$	difference
0.5			
1.5			
2.5			
4.0			

Discuss:

1. The network has zero training error. Has it learned the exact function?
 2. Why is performance on unseen inputs important?
-

Exercise 2.4 - What function has the ReLU network learned?

The fitted network is

$$\hat{f}(x) = 3 \text{ReLU}(x) - 4 \text{ReLU}(x - 1.5)$$

Because ReLU changes its behavior at zero, the expression changes at

$$x = 0$$

and

$$x = 1.5$$

Consider the three regions separately and determine $\hat{f}(x)$. Write the final result as

$$\hat{f}(x) = \begin{cases} \dots, & x < 0, \\ \dots, & 0 \leq x < 1.5, \\ \dots, & x \geq 1.5 \end{cases}$$

Plot the function. What is the most important visual difference between the quadratic target function and the ReLU network?

Solutions

Solution 2.1

The network is

$$h_1 = \text{ReLU}(x)$$

$$h_2 = \text{ReLU}(x - 1.5)$$

$$\hat{f}(x) = 3h_1 - 4h_2$$

Thus,

$$w_1 = 1, \quad b_1 = 0$$

$$w_2 = 1, \quad b_2 = -1.5$$

$$v_1 = 3, \quad v_2 = -4, \quad c = 0$$

Solution 2.2

At the training points:

x	$f(x)$	$\hat{f}(x)$
0	0	0
1	3	3
2	4	4
3	3	3

Therefore,

$$\text{MSE}_{\text{train}} = 0$$

Solution 2.3

However, at new inputs:

x	true $f(x)$	network $\hat{f}(x)$	difference $\hat{f} - f$
0.5	1.75	1.50	-0.25
1.5	3.75	4.50	0.75
2.5	3.75	3.50	-0.25
4.0	0.00	2.00	2.00

Thus, zero training error does not imply that the exact underlying function has been learned.

Solution 2.4

For

$$x < 0$$

both ReLUs are zero, so

$$\hat{f}(x) = 0$$

For

$$0 \leq x < 1.5$$

we have

$$\text{ReLU}(x) = x$$

and

$$\text{ReLU}(x - 1.5) = 0$$

Therefore,

$$\hat{f}(x) = 3x$$

For

$$x \geq 1.5$$

both ReLUs are active:

$$\hat{f}(x) = 3x - 4(x - 1.5)$$

Thus,

$$\hat{f}(x) = 3x - 4x + 6 = 6 - x$$

The complete function is

$$\hat{f}(x) = \begin{cases} 0, & x < 0, \\ 3x, & 0 \leq x < 1.5, \\ 6 - x, & x \geq 1.5 \end{cases}$$

The true function is quadratic, while the ReLU network is piecewise linear.

The locations of the changes in slope are determined by the points at which the pre-activations of the hidden ReLU neurons cross zero.

Additional hidden ReLU neurons can introduce additional changes in slope and therefore allow a finer piecewise-linear approximation.

Part III - A deeper neural network

So far, we have considered a neural network with one hidden layer. We now investigate a deeper network with the architecture

$$1 \rightarrow 3 \rightarrow 3 \rightarrow 1$$

There is

- one input
- a first hidden layer with three neurons
- a second hidden layer with three neurons
- one output neuron

Both hidden layers use the *ReLU* activation function.

In this part, the parameters are already given. You do not need to perform backpropagation for this larger network. The goal is to understand how information is propagated through multiple hidden layers and how additional neurons can change the shape of the learned function.

Exercise 3.1 - Architecture and number of parameters

For a fully connected neural network, every neuron in one layer is connected to every neuron in the next layer.

1. Draw the architecture

$$1 \rightarrow 3 \rightarrow 3 \rightarrow 1$$

2. Count the number of weights and biases between the input and the first hidden layer.
3. Count the number of weights and biases between the first and second hidden layers.
4. Count the number of weights and biases between the second hidden layer and the output.
5. Determine the total number of trainable parameters.

Compare this number with the seven trainable parameters of the network from Parts I and II.

Exercise 3.2 - First hidden layer

The first hidden layer is defined by

$$z_{1,i}^{(1)} = x_i, \quad h_{1,i}^{(1)} = \text{ReLU}(z_{1,i}^{(1)}),$$

$$z_{2,i}^{(1)} = x_i - 1, \quad h_{2,i}^{(1)} = \text{ReLU}(z_{2,i}^{(1)}),$$

and

$$z_{3,i}^{(1)} = x_i - 2, \quad h_{3,i}^{(1)} = \text{ReLU}(z_{3,i}^{(1)}).$$

Calculate the activations for the four training inputs.

x_i	$z_{1,i}^{(1)}$	$h_{1,i}^{(1)}$	$z_{2,i}^{(1)}$	$h_{2,i}^{(1)}$	$z_{3,i}^{(1)}$	$h_{3,i}^{(1)}$
0						
1						
2						
3						

At which input value does each of the three neurons become active?

How does this differ from the two-neuron hidden layer considered previously?

Exercise 3.3 - Second hidden layer

The second hidden layer receives the three outputs of the first hidden layer as its inputs.

It is defined by

$$z_{1,i}^{(2)} = h_{1,i}^{(1)}, \quad h_{1,i}^{(2)} = \text{ReLU}(z_{1,i}^{(2)})$$

$$z_{2,i}^{(2)} = h_{1,i}^{(1)} - h_{2,i}^{(1)}, \quad h_{2,i}^{(2)} = \text{ReLU}(z_{2,i}^{(2)})$$

and

$$z_{3,i}^{(2)} = h_{2,i}^{(1)} - h_{3,i}^{(1)}, \quad h_{3,i}^{(2)} = \text{ReLU}(z_{3,i}^{(2)})$$

Use the results from Exercise 3.2 and complete the table.

x	$z_{1,i}^{(2)}$	$h_{1,i}^{(2)}$	$z_{2,i}^{(2)}$	$h_{2,i}^{(2)}$	$z_{3,i}^{(2)}$	$h_{3,i}^{(2)}$
0						
1						
2						
3						

Notice that the second hidden layer no longer acts directly on x . It transforms features that were already created by the first hidden layer.

Exercise 3.4 - Output of the deep network

The output layer is

$$\hat{f}_{\text{deep}}(x) = -h_1^{(2)} + 4h_2^{(2)} + 2h_3^{(2)}.$$

Calculate the prediction for every training point.

x_i	true $f(x_i)$	deep network $\hat{f}_{\text{deep}}(x_i)$
0	0	
1	3	
2	4	
3	3	

Calculate the training MSE.

Compare the result with the fitted $1 \rightarrow 2 \rightarrow 1$ network from Part II.

- Do both networks achieve the same training error?
- Does this imply that both networks represent the same function?

Exercise 3.5 - Test the network between the training points

The two networks both reproduce the four training points exactly. We now compare what they predict between the training points.

Evaluate

$$f(x) = -x^2 + 4x$$

at

$$x = 0.5, 1.25, 1.75, 2.5$$

and use your result from the shallow network of Part II. For the deep network, perform the forward pass through both hidden layers and complete the table.

x_i	true $f(x_i)$	shallow network	deep network
0.5			
1.25			
1.75			
2.5			

Which network is closer to the true function at these points? Why can two networks with zero training error behave differently between the training observations?

Exercise 3.6 - What function has the deeper network learned?

Determine the output of the deeper network in the following regions:

1. $x < 0$
2. $0 \leq x < 1$
3. $1 \leq x < 2$
4. $x \geq 2$

Write the result as a piecewise function

$$\hat{f}_{\text{deep}}(x) = \begin{cases} \dots, & x < 0, \\ \dots, & 0 \leq x < 1, \\ \dots, & 1 \leq x < 2, \\ \dots, & x \geq 2 \end{cases}$$

Plot in the same coordinate system

- the true function $f(x) = -x^2 + 4x$
- the four training points
- the fitted $1 \rightarrow 2 \rightarrow 1$ network from Part II
- the fitted $1 \rightarrow 3 \rightarrow 3 \rightarrow 1$ network

What has changed when moving to the larger network?

In particular, compare the number and positions of the changes in slope.

Solutions

Solution 3.1

For the architecture

$$1 \rightarrow 3 \rightarrow 3 \rightarrow 1$$

the number of parameters can be counted layer by layer. Between the input and the first hidden layer there are

$$1 \cdot 3 = 3$$

weights and 3 biases. This gives

$$3 + 3 = 6$$

parameters.

Between the first and second hidden layers there are

$$3 \cdot 3 = 9$$

weights and 3 biases. This gives

$$9 + 3 = 12$$

parameters.

Between the second hidden layer and the output there are

$$3 \cdot 1 = 3$$

weights and 1 bias. This gives

$$3 + 1 = 4$$

parameters.

Thus, the complete network contains

$$6 + 12 + 4 = 22$$

trainable parameters.

The smaller network from Parts I and II contains only 7 trainable parameters. The deeper network therefore has considerably more degrees of freedom.

Solution 3.2

The first hidden layer is

$$h_1^{(1)} = \text{ReLU}(x)$$

$$h_2^{(1)} = \text{ReLU}(x - 1)$$

and

$$h_3^{(1)} = \text{ReLU}(x - 2)$$

For the four training points:

x_i	$z_{1,i}^{(1)}$	$h_{1,i}^{(1)}$	$z_{2,i}^{(1)}$	$h_{2,i}^{(1)}$	$z_{3,i}^{(1)}$	$h_{3,i}^{(1)}$
0	0	0	-1	0	-2	0
1	1	1	0	0	-1	0
2	2	2	1	1	0	0
3	3	3	2	2	1	1

The three neurons change their behavior at

$$x = 0, \quad x = 1, \quad x = 2$$

Thus, the first hidden layer introduces three different ReLU thresholds.

Compared with the previous two-neuron hidden layer, more hidden neurons allow the network to create more distinct regions in which the overall slope may change.

Solution 3.3

The second hidden layer is defined by

$$h_1^{(2)} = \text{ReLU}(h_1^{(1)})$$

$$h_2^{(2)} = \text{ReLU}(h_1^{(1)} - h_2^{(1)})$$

and

$$h_3^{(2)} = \text{ReLU} \left(h_2^{(1)} - h_3^{(1)} \right)$$

Using the results from the first hidden layer gives

x_i	$z_{1,i}^{(2)}$	$h_{1,i}^{(2)}$	$z_{2,i}^{(2)}$	$h_{2,i}^{(2)}$	$z_{3,i}^{(2)}$	$h_{3,i}^{(2)}$
0	0	0	0	0	0	0
1	1	1	1	1	0	0
2	2	2	1	1	1	1
3	3	3	1	1	1	1

The important difference from the first hidden layer is that these neurons no longer receive the original input x directly.

Instead, they receive the features

$$h_1^{(1)}, \quad h_2^{(1)}, \quad h_3^{(1)}$$

that were constructed by the previous layer.

The second hidden layer therefore combines already transformed information.

Solution 3.4

The output is

$$\hat{f}_{\text{deep}}(x) = -h_1^{(2)} + 4h_2^{(2)} + 2h_3^{(2)}$$

Using the activations from Exercise 3.3:

x_i	true $f(x_i)$	$h_1^{(2)}$	$h_2^{(2)}$	$h_3^{(2)}$	deep network $\hat{f}_{\text{deep}}(x_i)$
0	0	0	0	0	0
1	3	1	1	0	3
2	4	2	1	1	4
3	3	3	1	1	3

Thus, the training MSE is

$$\text{MSE}_{\text{train}} = 0$$

The shallow network from Part II also has zero training error.

However, this does not imply that both networks represent the same function. The training loss only compares the predictions at the four training points. The networks may behave differently between or outside these points.

Solution 3.5

The true target function is

$$f(x) = -x^2 + 4x$$

The true values are

$$f(0.5) = 1.75$$

$$f(1.25) = 3.4375$$

$$f(1.75) = 3.9375$$

and

$$f(2.5) = 3.75$$

For the shallow network,

$$\hat{f}_{\text{shallow}}(x) = \begin{cases} 0, & x < 0, \\ 3x, & 0 \leq x < 1.5, \\ 6 - x, & x \geq 1.5 \end{cases}$$

Therefore,

$$\hat{f}_{\text{shallow}}(0.5) = 1.5$$

$$\hat{f}_{\text{shallow}}(1.25) = 3.75$$

$$\hat{f}_{\text{shallow}}(1.75) = 4.25$$

and

$$\hat{f}_{\text{shallow}}(2.5) = 3.5$$

For the deeper network, the forward pass gives

$$\hat{f}_{\text{deep}}(0.5) = 1.5$$

$$\hat{f}_{\text{deep}}(1.25) = 3.25$$

$$\hat{f}_{\text{deep}}(1.75) = 3.75$$

and

$$\hat{f}_{\text{deep}}(2.5) = 3.5$$

The comparison is therefore

x_i	true $f(x_i)$	shallow network	deep network
0.5	1.7500	1.5000	1.5000
1.25	3.4375	3.7500	3.2500
1.75	3.9375	4.2500	3.7500
2.5	3.7500	3.5000	3.5000

At $x = 0.5$ and $x = 2.5$, both networks have the same error.

At $x = 1.25$ and $x = 1.75$, the deeper network is closer to the true quadratic function.

Thus, both networks have zero training error, but they interpolate the training observations differently.

Solution 3.6

For

$$x < 0$$

all first-layer ReLUs are zero. Therefore,

$$\hat{f}_{\text{deep}}(x) = 0$$

For

$$0 \leq x < 1$$

we have

$$h_1^{(1)} = x, \quad h_2^{(1)} = 0, \quad h_3^{(1)} = 0$$

Therefore,

$$h_1^{(2)} = x, \quad h_2^{(2)} = x, \quad h_3^{(2)} = 0$$

and

$$\hat{f}_{\text{deep}}(x) = -x + 4x = 3x$$

For

$$1 \leq x < 2$$

we have

$$h_1^{(1)} = x, \quad h_2^{(1)} = x - 1, \quad h_3^{(1)} = 0$$

Thus,

$$h_1^{(2)} = x$$

$$h_2^{(2)} = \text{ReLU}(x - (x - 1)) = 1$$

and

$$h_3^{(2)} = \text{ReLU}(x - 1) = x - 1$$

Therefore,

$$\hat{f}_{\text{deep}}(x) = -x + 4 + 2(x - 1) = x + 2$$

For

$$x \geq 2$$

we have

$$h_1^{(1)} = x, \quad h_2^{(1)} = x - 1, \quad h_3^{(1)} = x - 2$$

Thus,

$$h_1^{(2)} = x, \quad h_2^{(2)} = 1, \quad h_3^{(2)} = 1$$

Therefore,

$$\hat{f}_{\text{deep}}(x) = -x + 4 + 2 = 6 - x$$

The complete function is

$$\hat{f}_{\text{deep}}(x) = \begin{cases} 0, & x < 0, \\ 3x, & 0 \leq x < 1, \\ x + 2, & 1 \leq x < 2, \\ 6 - x, & x \geq 2 \end{cases}$$

The shallow network changes slope at

$$x = 0$$

and

$$x = 1.5$$

The deeper network changes slope at

$$x = 0, \quad x = 1, \quad x = 2$$

Thus, the deeper network uses more linear regions and gives a finer piecewise-linear approximation of the quadratic target function.

This does not mean that a deeper or wider network must always generalize better. It only means that the larger network has greater representational flexibility.

Short conceptual questions

Question 1

Why is the output of this ReLU network piecewise linear?

Question 2

What determines the positions at which the slope changes?

Question 3

What role do the output weights v_1 and v_2 play?

Question 4

Why can a network with only two hidden neurons already represent a function that is more complicated than a single straight line?

Question 5

What would additional ReLU neurons allow the network to do?

Question 6

Why does zero training error not guarantee good generalization?

Solutions - Short conceptual questions

Solution - Question 1

A ReLU function is piecewise linear:

$$\text{ReLU}(z) = \begin{cases} 0, & z < 0, \\ z, & z \geq 0 \end{cases}$$

The transformations before and after the ReLU functions are linear or affine. Combining affine transformations with piecewise-linear ReLU activations therefore produces another piecewise-linear function.

Solution - Question 2

The slope changes when a hidden ReLU neuron changes between its inactive and active state.

For a neuron

$$z = wx + b$$

this happens when

$$wx + b = 0$$

Thus, the weights and biases determine where the activation boundaries occur. In deeper networks, later layers act on activations created by earlier layers, so additional changes in slope can also result from combinations of previously constructed features.

Solution - Question 3

The output weights determine how strongly each hidden-neuron activation contributes to the final prediction.

For example,

$$\hat{y} = v_1 h_1 + v_2 h_2 + c$$

The magnitude of v_j scales the contribution of neuron j , while its sign determines whether that contribution increases or decreases the output.

Therefore, the output weights strongly influence the slopes and overall shape of the final piecewise-linear function.

Solution - Question 4

A single affine neuron can only represent one straight line.

A hidden ReLU neuron can introduce a change in slope at the point where it becomes active. With two hidden neurons, the network can combine two such activations and therefore construct several linear regions with different slopes.

This makes the resulting function more flexible than a single straight line.

Solution - Question 5

Additional ReLU neurons can introduce additional activation thresholds and additional learned features.

This can allow the network to create more linear regions and approximate a nonlinear target function more closely.

However, more neurons also increase the number of trainable parameters and the model capacity. More neurons therefore do not automatically guarantee better predictions on unseen data.

Solution - Question 6

Zero training error only means that the network reproduces the observed training targets.

A finite training dataset does not uniquely determine the function between or outside the observed points. Many different functions can agree on all training observations while making very different predictions elsewhere.

Therefore,

zero training error does not imply good generalization

Performance on unseen data must be evaluated separately.